



HUST



26/06/2026

Projet Smart système 4A ISIE

Federated Learning for Vehicle and Pedestrian Detection on Embedded Systems

University of Tours

École Polytechnique de l'université de Tours

Computing and Embedded Intelligent Systems

Etudiants : Hugo BRAY / Aurelien PAPILLON

Tutor: Mr. Pierre GAUCHER & Ph.D Thanh-Hai Tran & Ph.D Hai vu

Table of Contents

Acknowledgement.....	4
Introduction	5
1 General presentation and scoping of the project	6
1.1 Analysis of needs and strategic objectives	6
1.2 Specifications and requirements of the subject	6
2 Designing the Distributed Architecture.....	9
2.1 Topology of the Federated Learning Network (FL).....	9
2.2 Centralized Learning: Establishing the Reference Model.....	10
2.3 Prototyping phase: from simulation to single customer.....	11
3 Data management and the learning cycle.....	13
3.1 Initial assessment and limitations of YOLOv8n	13
3.2 Technology migration to YOLOv5	13
4 Data management and the learning cycle.....	14
4.1 Evaluation and Analysis of Results	14
4.2 Implementing the Flower Protocol and the FedAvg Algorithm	14
5 Hardware Troubleshooting and Optimizations	18
5.1 Physical network start-up scenario.....	18
5.2 Analysis of hardware anomalies and crashes	18
6 Deployment, Evaluation and Analysis of Results	21
6.1 Technical solutions and configurations applied to embedded boards	21
6.2 Performance evaluation after optimization	21
7 Project Management Review	23
7.1 Monitoring, Time Planning and Milestones (Gantt)	23
7.2 Organization and communication	23
8 Conclusion.....	24

Acknowledgement

We would like to express our gratitude to all those who contributed to the realization of this project.

First of all, we would like to thank our supervisors, Mr. Pierre Gaucher, Ph. Thanh-Hai Tran and Ph. Hai Vu, for their availability, their sound advice and their follow-up. Their expertise was decisive in guiding our technical choices and in the success of this project.

We also thank the University of Tours and Polytech Tours for providing us with the framework and resources necessary to conduct this work, as well as the HUST Vietnam University for this enriching international collaboration that made it possible to deploy our system on its facilities.

Introduction

As part of our engineering training at Polytech Tours, in the Computer Science and Intelligent Embedded Systems program, we are conducting a project focused on distributed artificial intelligence technologies. This project, carried out in international collaboration with the Vietnamese University HUST, is entitled "*Federated Learning* for Vehicle and Pedestrian Detection on Embedded Systems".

The main objective of this work is to design, implement and evaluate a complete end-to-end *Federated Learning* (FL) system. Dedicated to the real-time detection of vehicles and pedestrians, this system aims to overcome the constraints of the classic centralized Cloud approach, particularly in terms of bandwidth saturation and respect for the confidentiality of private data. To address this issue, our research exploits the mechanisms of embedded computing (*Edge Computing*): raw data remains compartmentalized on the terminals and only a lightweight vision model trains collaboratively by exchanging its parameters with a central orchestration server.

On the technical side, the approach is based on detection algorithms from the YOLO family and a distributed network infrastructure, with the aim of deployment and evaluation on constrained hardware targets (Edge devices).

This report presents the approach adopted and the topology of our architecture. It details the progression of the project, from the initial computer simulation phases to the analysis of real performance on embedded boards (accuracy scores, inference latency, communication overhead and hardware resources). Finally, it draws up a comparative assessment between the centralised and federated approach, before highlighting the limits encountered and the prospects for the evolution of the system.

1 General presentation and scoping of the project

1.1 Analysis of needs and strategic objectives

Autonomous vehicles rely on their ability to perceive and understand their surroundings continuously and instantaneously. To ensure the critical safety of users, the on-board system must operate with low latency, as the slightest non-detection represents a potential risk of accident. As a result, the false negative rate must tend towards zero. This safety requirement requires the vehicle to locate and identify pedestrians as well as other vehicles in real time.

To train the artificial intelligence models necessary for this perception, the classic centralized approach on the Cloud currently faces two major obstacles. The first concerns bandwidth: a single autonomous vehicle generates several terabytes of data per day, making sending these raw streams to a Cloud server technically and economically unfeasible on a large scale. The second challenge concerns data privacy and security. Images captured in real-world conditions contain sensitive information such as faces, license plates, or precise location coordinates. Their centralization is therefore severely restricted by strict regulations as well as industrial privacy policies.

The central problem of this project can be formulated as follows: **how to train a high-performance detection model on data distributed between vehicles without ever centralizing the raw images on a server, and without degrading the overall performance of the system?**

The solution explored and provided throughout this work is in the implementation of *Federated Learning* coupled with the aggregation algorithm.

1.2 Specifications and requirements of the subject

The project topic was provided to us with a set of objectives, system components, and expected deliverables. This section presents a structured synthesis of these requirements, before explaining in the next section how each of these requirements has been translated into concrete technical choices.

1.2.1 Prioritizing Requirements: The Moscow Matrix

To set priorities from the beginning of the project, we classified the requirements of the specifications with the MoSCoW method (*Must have, Should have, Could have, Won't have*).

Vision Model: Deploy a YOLO algorithm on the boards to detect bicycles, cars, and pedestrians. Distributed Network: Create a federated learning architecture between the PC server and the edge boards (Jetson and Raspberry Pi). Data Privacy: Keep the KITTI images locally on the boards. No images must travel over the network. Data Fusion: Use the FedAvg algorithm to average the weights sent by the boards. MUST	AI Monitoring: Track mAP and Loss scores at each round to verify that the AI is learning correctly. Hardware Monitoring: Measure CPU usage, RAM usage, computation time, and the power consumption of the boards. Custom Training: Be able to configure a different number of local epochs from one board to another based on hardware performance. SHOULD
COULD Configuration File: Create a JSON file on the server to change hyperparameters without modifying the code. Fair Data Distribution: Split the images equally among the boards (quasi-IID approach).	WON'T Network Security: Encrypt the gRPC channel with SSL/TLS certificates (not enough time). Graphical Interface: Create a real-time visual application for the end user.

Alternative Algorithm: Test the FedProx algorithm to see if it handles the differences between the boards better.	
--	--

Figure1 : Matrice MoSCoW

1.2.2 Expected Contract Deliverables

To validate the success of this project, the final system must be accompanied by a series of specific deliverables:

- **A functional FL system:** A network infrastructure that works in a real and simulated environment.
- **A trained detection model:** A neural network optimized for vehicle and pedestrian recognition.
- **Inference Demonstration:** Visual proof of the proper functioning of the detection directly on the embedded Edge hardware.
- **A comprehensive technical report:** This document covers the system architecture, FL parameters, evaluation results, comparison between centralized and federated training, and analysis of limits and perspectives.

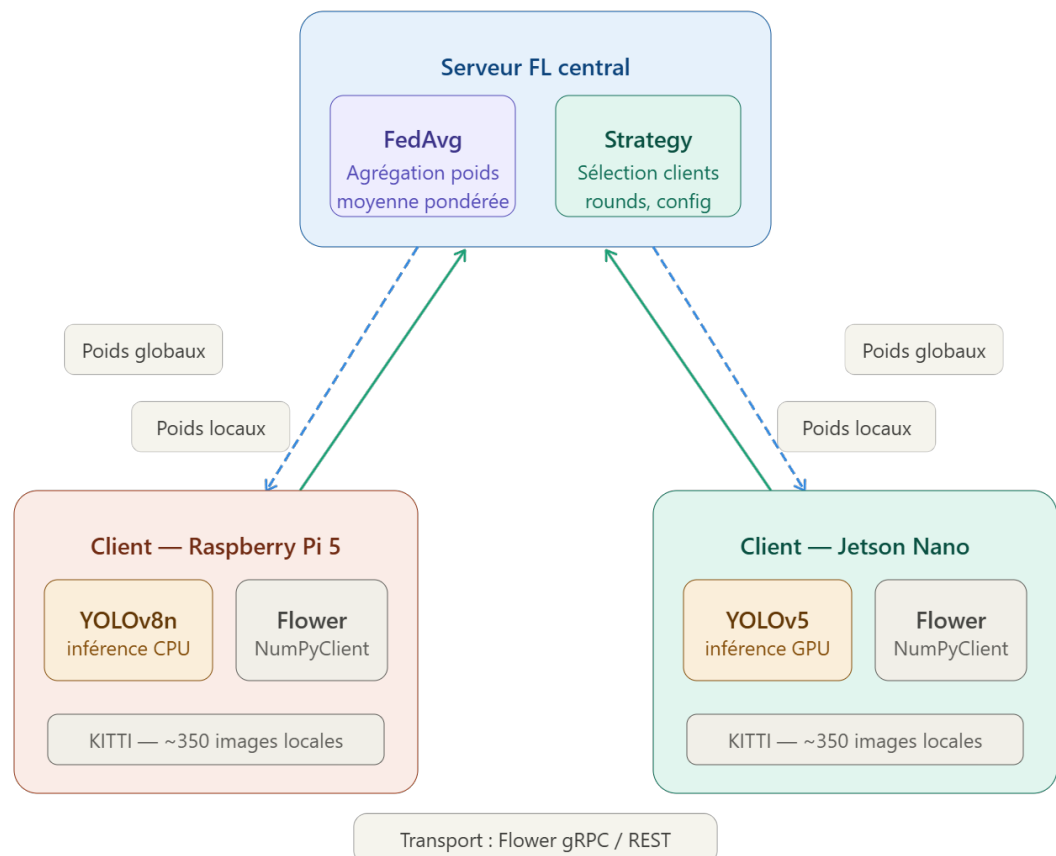
The main objective of this project is to design, implement and evaluate a complete end-to-end *Federated Learning* (FL) system. This system is dedicated to the real-time detection of vehicles and pedestrians, before being deployed on constrained hardware targets (*Edge devices*).

2 Designing the Distributed Architecture

2.1 Topology of the Federated Learning Network (FL)

The system developed is based on a *Federated Learning* (FL) architecture at two levels: a central server in charge of model aggregation and three embedded clients. This organization makes it possible to train object detection models without centralizing the raw data.

- **FL Server:** It runs on a local computer and orchestrates the entire process based on the Flower framework. Each round, it manages the connection and participation of the cards, distributes the global model weights to the connected clients, waits for their updated local weights to be received, and then applies the aggregation algorithm to generate the global model.
- **Embedded boards (FL clients):** Composed of Raspberry Pi and Jetson Nano modules, these hardware nodes are connected to the server in a local network. Each map is responsible for executing the model's local training on its own local data.



2.2 Centralized Learning: Establishing the Reference Model

Before designing the distributed infrastructure, we started the project with a phase of understanding and getting started with the YOLO ecosystem through a centralized learning approach.

This preliminary step consisted of grouping the entire subset of KITTI data (i.e. the 700 annotated images) on a single high-performance physical machine to train the vision model locally.

The objective of this manipulation was twofold:

Validate the detection pipeline: Ensure that the annotation format, hyperparameters of the YOLO architecture were fully functional and error-free.

Define a point of comparison: Get the theoretical maximum scores (mAP and convergence speed) of the model when it has direct and unrestricted access to all data. These "centralized" metrics served as an absolute reference to evaluate, subsequently, the loss or maintenance of efficiency induced by the compartmentalization of federated learning data.

For this initial centralized learning phase, we chose the **YOLOv8n** model (*nano* version). This choice is justified by the excellent theoretical compromise offered by this latest-generation architecture between an extremely small model size — ideal for the storage constraints of future embedded systems — and a high inference speed, which is essential to guarantee real-time vehicle and pedestrian detection without compromising overall accuracy.

2.3 Prototyping phase: from simulation to single customer

The development of the system followed a rigorous progression in phases to validate each layer of complexity.

Phase 1: Computer simulation

This first step was to simulate the entire FL system on a single computer, with no onboard hardware, to ensure that the federated drive logic was correct. Specifically, two Flower customers were instantiated as separate processes on the same machine. Each client had half of the dataset (700 images each). This phase made it possible to validate:

- Local training logic and weight transmission to the server.
- The proper functioning of the FedAvg algorithm (weighted average).
- The mathematical coherence of merging weights without network constraints.

Phase 2: Deployment on Raspberry Pi 5 (single client)

The second step marked the transition to real hardware by connecting a first embedded board (the Raspberry Pi 5) to the Flower server running on the PC. For this phase, the YOLOv8 model was used to test inference and learning on a physical target.

Using a single client on the local network made it easier to find errors and stabilize connections. This phase has provided us with several lessons and validations:

- **Getting started with Flower and YOLO:** It made it possible to discover and validate simple communications with Flower, as well as to concretely test the behavior of a YOLO model directly on the Raspberry Pi architecture.
- **Remote Control (SSH):** This was an opportunity to discover and configure remote control of the card via SSH links from a host PC, making it easy to run scripts without a dedicated screen.
- **Hardware performance:** We noticed that the execution speed of the Raspberry Pi 5 is remarkable for embedded devices, especially thanks to the power of its new processor.
- **Memory management (Workers):** In an effort to optimize data loading, we tested the maximum number of parallel processes (*workers*). This caused our first crashes for *Out of Memory*, which allowed us to understand the limitations of the card's RAM configuration.
- **Overheating Fix:** During HPC phases, we encountered our first overheating issues on the board. This point has been corrected by installing the Raspberry Pi in an official ventilated case.
- **Power supply adjustment:** We found a lack of electrical power when using a typical USB Type-C cable (the board throttled its performance or displayed alerts). We solved this problem by using the official Raspberry Pi power supply capable of delivering 5A at most.

3 Data management and the learning cycle

3.1 Initial assessment and limitations of YOLOv8n

Initially, we chose the YOLOv8n model for its optimal theoretical performance, offering a small size and an excellent compromise between speed and precision. As detailed previously in Phase 2 (Section 2.3), the Raspberry Pi 5 served as a technological "sandbox".

This milestone was strategic because it allowed us to decouple the problems: before confronting the system with the heterogeneity of the network, we used YOLOv8n to validate the robustness of the Flower communication protocol. This step proved that the gRPC architecture was able to support the weight transfer of a modern model, while allowing us to define our thermal and electrical management standards under real inference conditions

3.2 Technology migration to YOLOv5

However, during Phase 3 aimed at extending the network to all physical targets (integration of the two NVIDIA Jetson Nano cards), a critical technical lock appeared.

The official Jetson Nano system environment (**JetPack 4.6.1**) mandates a fixed version of Python at **3.6.9**. However, the official library of YOLOv8 (ultralytics) requires Python 3.8. In addition, the support for graphics acceleration (CUDA 10.2) on the Jetson Nano is strictly related to specific and fixed versions of $\geq$ **PyTorch (1.10)** and **Torchvision**, which are incompatible with the software requirements of recent versions of YOLOv8. All our attempts to force the

installation of YOLOv8 or to update the Python environment have broken compatibility with CUDA drivers, making it impossible to use the GPU.

Faced with this insurmountable material constraint, we have made a **technological pivot to YOLOv5**. This model is fully compatible with Python 3.6.9 as well as the PyTorch 1.10 / CUDA 10.2 combination provided by NVIDIA for the Jetson Nano. YOLOv5 offers performance that is perfectly suited for vehicle and pedestrian detection. To maintain the homogeneity of the learning network, the Raspberry Pi 5 has also been switched to YOLOv5. This choice ensured seamless integration, full access to GPU hardware acceleration, and functional execution across all of our heterogeneous targets.

4 Data management and the learning cycle

4.1 Evaluation and Analysis of Results

Validation and evaluation of system performance is based on the KITTI reference dataset. In order to simulate stable learning conditions, the data volume was segmented in a quasi-IID (*Independent and Identically Distributed*) manner.

The dataset is thus partitioned equally between the three active physical nodes of the network (the Raspberry Pi and the two Jetson Nano), which represents 200 training images per device. This choice respects the fundamental philosophy of *Federated Learning*, where raw data remains stored locally on each *edge device*.

To efficiently manage the distribution of these batches of images and automate their transfer to the various embedded boards, we have designed command pipelines combining the "find", "head" and "rsync" tools.

4.2 Implementing the Flower Protocol and the FedAvg Algorithm

4.2.1 Configuring Network Infrastructure with Flower

The software implementation relies on Flower for gRPC network and cycle management. On the server side, the policy is instantiated with the following control settings:

fraction_fit = 1.0: All available customers participate in each training round.

Fraction_evaluate = 0: Systematic evaluation at the end of each round.

min_available_client = 3 : The server blocks and waits for both Jetson Nano ("comvis" and "js1") and the Raspberry Pi to be connected before starting the first round.

4.2.2 Algorithmic Logic of Global Fusion (FedAvg)

The algorithmic core of the merge is based on a custom Python script (fusion.py) that implements the **FedAvg** (*Federated Averaging*) aggregation logic. At the end of each round, customers transmit their updated local weights in the form of NumPy tables as well as their associated metrics (loss, number of examples). The server then takes a weighted average, layer by layer, of these synaptic weights in proportion to each client's data volume, before redistributing the updated global model.

The following table summarizes the final configuration of our test bench:

Component	Role	Model	Framework	Dataset local
Server (PC)	Agrégation FL	—	Flower + FedAvg	—
Jetson Nano comvis	Client FL 1	YOLOv5	Flower + PyTorch	200 images KITTI
Jetson Nano js1	Client FL 2	YOLOv5	Flower + PyTorch	200 images KITTI
pi_01	Client FL 3	YOLOv5	Flower + PyTorch	200 images KITTI

Table 1 – Federated Learning System Configuration Deployed on Real Hardware

4.2.3 Dynamic control via server configuration file

To avoid freezing hyperparameters in the source code and to enable agile network management, we have set up a centralized configuration file in JSON format ("confid_server.json")

on the server side. This file allows the network administrator to dynamically define the conditions of the federated drive for each client individually:

The number of global rounds: Defines the total cycle of iterations of federated learning.

Minimum number of clients required: Specifies the threshold of connected adapters needed for the server to officially start the learning process (thus avoiding starting if a device is offline).

The number of local epochs (epochs): Specifies the number of practice passes that each card must perform on its own GPU or CPU before sending its weights to the server.

Batch size (batch_size): Adjusts the volume of images processed simultaneously during a pass, a critical parameter for controlling and avoiding memory saturation on the embedded system.

Number of sub-processes (workers): Controls the allocation of CPU resources for parallel loading of images.

This file also includes a "default" entry. It ensures system scalability: if a new, unknown client connects to the gRPC network, the server automatically assigns these metrics to the gRPC network, without requiring the server to be shut down or scripts to be changed.

4.2.4 Execution protocol and sales order arguments

Training on Edge devices is initiated via a Python script ("client_jetson.py" or "client_pi.py") executed from the command line. In order to make the implementation generic and adaptable to each node, we have developed a modular argument system.

A typical command looks like this:

```
python3 client_jetson.py --data /home/comvis/kitti/data.yaml --server 192.168.0.10X:8080 --device-type jetson --device-id comvis
```

Each attribute plays a specific role in initializing the node:

--data : Specifies the absolute path to the dataset's configuration file (data.yaml). This allows the map to locate its local images and to know the number of classes to be detected (here, reduction from 80 to 8 classes for the adaptation to KITTI).

--server : Specifies the IP address and listening port of the central Flower server (IP:PORT). It is this target that the gRPC channel will try to reach to register in the round.

--device-type : Specifies the nature of the hardware (jetson or raspberrypi). This argument is crucial because it allows the script to automatically configure the hardware environment, including enabling cuda :0 graphics acceleration on the Jetson or by forcing the execution on cpu for the Raspberry Pi.

--device-id : Assigns a unique identifier to the client (comvis, js1, etc.). This identifier allows the server to formally recognize the adapter that is communicating with it and to apply its custom configuration from the JSON file to it.

5 Hardware Troubleshooting and Optimizations

5.1 Physical network start-up scenario

Once the YOLOv5 and Flower environment were stabilized, we proceeded with the actual deployment. The protocol consisted of running the server script on the PC, opening a gRPC communication port, and then running the client scripts simultaneously on the embedded boards.

5.2 Analysis of hardware anomalies and crashes

As soon as the first rounds of training were launched on the map network, we were faced with two critical outages that were blocking the system:

5.2.1 RAM Full-Throttle ("Cannot allocate memory" error)

On Jetson Nano boards, the client script managed to register with the gRPC server but crashed instantly when initiating the first epoch of local training, returning the systematic error: `OSError: [Errno 12] Cannot allocate memory`. The in-depth analysis of system resources revealed two major causes:

- **Memory fragmentation:** At rest, JetPack's graphics environment and core services already consume a significant portion of the Jetson's 4GB of RAM. After several tests or abrupt interruptions of scripts, the memory fragments.
- **Insufficient ZRAM mechanism:** By default, the JetPack architecture configures a RAM-compressed virtual SWAP space (ZRAM) limited to 1.9 GB. When loading the

YOLOv5 model into memory and allocating the data load sub-processes (*workers*), this barrier is immediately breached, causing an *Out Of Memory (OOM)* crash.

5.2.2 Electrical instability and hardware disconnections

On the Jetson "comvis", the card simply disconnected from the local network (instant loss of SSH links and the Flower session) a few seconds after the start of the training. This phenomenon is linked to a critical drop in electrical voltage. At idle, the Jetson has a moderate consumption of 2 to 3W. As soon as the CUDA GPU and CPU are used at 100% to calculate the gradients of YOLOv5, the energy demand instantly climbs beyond 10W. As the power supply via the classic Micro-USB port is unable to handle this current peak, the voltage collapses and the card goes into hardware security by cutting off its interfaces or rebooting.

5.2.3 Recurring link losses related to network infrastructure (Router)

Apart from purely energy outages, the test bench suffered from very frequent gRPC disconnections at the network layer. The router used for local interconnect struggled to handle maintaining the persistent asynchronous connections required by the Flower framework. During the local computing phases where the gRPC channel remained "silent" (the time it took for the cards to finish their epochs), the router would prematurely close the sockets or suffer micro-cuts. These instabilities caused the server to immediately exclude clients, forcing the current federated round to be completely interrupted.

5.2.4 Native software heterogeneity between Raspberry Pi and Jetson Nano

The simultaneous integration of heterogeneous targets highlighted a major conflict of native software versions:

The Raspberry Pi 5 has a recent operating system that natively provides a modern version of Python (3.10), which initially allowed YOLOv8 to run smoothly.≥

Conversely, the Jetson Nano is limited by its JetPack 4.6.1 manufacturer architecture, which imposes an obsolete and fixed version of Python 3.6.9.

This deep divergence made it impossible to use a single, standardized customer code. The official library of YOLOv8 (ultralytics) categorically refuses to install on Python 3.6, and the CUDA GPU acceleration (10.2) of the Jetson being intimately linked to fixed versions of PyTorch (1.10) and Torchvision, we were confronted with a real wall of compatibility.

5.2.5 Complexity of replicating the environment on a new Jetson

When introducing a second Jetson Nano to expand our network to three customers, setting up the solution proved to be particularly laborious. Reconstituting exactly the same software ecosystem (strict alignment of the versions of each library, Flower, PyTorch, Torchvision and image processing dependencies) represents a complex challenge on these constrained architectures.

To bypass the hell of global dependencies and ensure perfect mirror alignment with the first Jetson, we had to isolate the application through a virtual environment (venv). The installation required manually compiling some Python *wheels* specific to NVIDIA's ARM64 architecture, to ensure that the algorithmic behavior and structure of the weights exported as NumPy arrays remained rigorously identical.

6 Deployment, Evaluation and Analysis of Results

6.1 Technical solutions and configurations applied to embedded boards

To resolve the Chapter 5 locks and stabilize our benchmark, we applied three major optimizations:

- **Creating a Persistent Physical SWAP:** We allocated a 4 GB swap file directly to the Jetson Nano's SD card to support the RAM. To prevent this configuration from being erased every time it is shut down or rebooted, we have permanently written it to the `/etc/fstab` system file descriptor via the command: `echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab`. At idle, the `free -h` command validates an extended total available memory (RAM + SWAP) and a RAM used stabilized at only 1.0 GB.

6.2 Performance evaluation after optimization

Once the equipment was stabilized, we conducted a full training. The performances were recorded in a CSV file in each round.

6.2.1 Data collection pipeline and CSV reporting

The performance analysis presented in this chapter is not based on theoretical estimates, but on the use of real data collected directly in the field at each learning cycle.

To do this, we instrumented our client scripts to measure and extract physical and algorithmic metrics at the end of each Flower round. On the client side, a monitoring module continuously records:

- **AI performance metrics:** The value of the *local loss* and the mAP score calculated on the card validation set.
- **Hardware metrics:** The exact local drive time (in seconds), the average CPU utilization, the amount of RAM consumed, as well as the volume of network data transited (bytes sent and received via gRPC).

At the end of each round, each customer structures this data and exports it autonomously. This information is uploaded and centralized on the server in a history file in CSV format. This consolidated metrics file is our analytical database. It is from this document that we were able to draw the convergence curves and draw up the comparative CPU/GPU efficiency tables presented below.

6.2.2 Evolution of AI metrics (mAP and Loss)

The analysis shows a consistent progression of the overall model:

- The mAP@50 has steadily increased over the rounds, going from an average of 0.0245 (round 1) to 0.0708 (round 4), an improvement of **+189%**. The values of mAP@50-95 follow the same trend, from 0.0056 to 0.0250. Although these absolute values remain low (which can be explained by a small number of rounds tested, only one local epoch per round epochs=1 and a dataset restricted to 700 images), the curve mathematically validates the convergence of federated learning.
- The loss decreases from 0.1974 (round 1) to 0.1419 (round 3), a reduction of **-28.1%**. The loss difference between the two Jetsons in each round is almost zero (0.0001), which validates the consistency of the FedAvg algorithm on identical client configurations.<

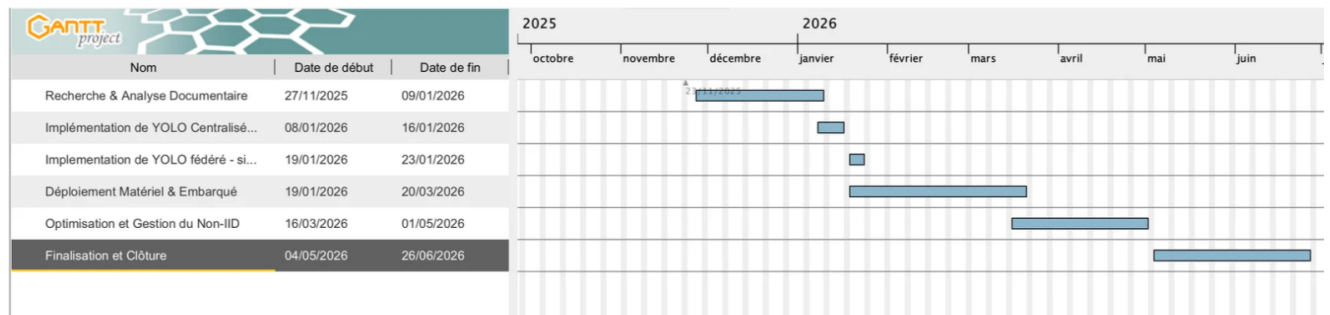
6.2.3 Hardware behavior and performance

The average CPU usage of the Jetson Nano remained moderate (between 13% and 21%). RAM occupancy has stabilized between 1,350 and 2,382 MB depending on the phase. However, there is a temporal heterogeneity: the "comvis" map has a systematically higher local training time than the "js1" map (730–806 s compared to 451–512 s). This is most likely due to the cooling of the "comvis" which is much more efficient.

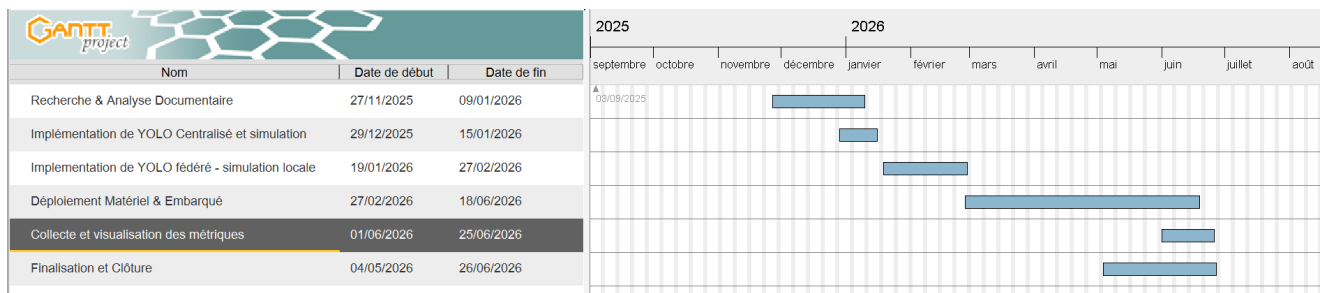
7 Project Management Review

7.1 Monitoring, Time Planning and Milestones (Gantt)

Regarding the project management of the project, we have set up, at the beginning of the project, a Gantt to manage the schedule (below).



We can compare the Gantt achieved at the beginning of the project as well as the one actually achieved during the project, we observe that they are different, in particular due to the problems encountered at the version level during the implementation of phase 3.



7.2 Organization and communication

In order to ensure transparency and good management of the project, we maintained regular communication with our tutor through periodic follow-up meetings. These exchanges were an opportunity to present the progress of our work and to validate the main stages of the project.

In addition, the frequent writing of progress reports has allowed us to formalize the follow-up of tasks (completed and planned), thus ensuring that our Gantt is respected as much as possible.

8 Conclusion

This project, carried out in collaboration with Vietnamese HUST University, created and tested a comprehensive Federated Learning system for detecting vehicles and pedestrians. This Edge Computing approach eliminates the need to send raw images to a cloud server, which solves bandwidth and privacy issues. The data from the KITTI dataset thus remains stored locally on the maps. We validated the training logic and weight fusion with the FedAvg algorithm first in simulation and then on a physical network consisting of a central server and three clients: a Raspberry Pi 5 and two NVIDIA Jetson Nano. The server orchestrates the entire process through the Flower framework and a custom script called `fusion.py`.

The transition to real equipment highlighted several physical difficulties specific to embedded systems. The intensive training required hardware corrections to manage the overheating and voltage drops of the cards. To fix the memory (RAM) crashes on the Jetson Nano, we had to add a SWAP file. In addition, the Jetson Nano's JetPack 4.6.1 system blocks Python in version 3.6.9, which prevented the use of YOLOv8. So we chose to switch the entire network to YOLOv5. This choice made it possible to use GPU graphics acceleration with CUDA while keeping an identical model between the Raspberry Pi and the Jetson for the fusion of weights to work.

The results show that the overall model is improving, as evidenced by the decrease in *loss* and the increase in *mAP@50* accuracy over the rounds. For the rest of the project, several important perspectives emerge. First, it would be helpful to create an automated installation package to install all libraries and configurations at once on a new board. Secondly, the use of newer Jetson boards would allow for a modern version of Python and thus switch to a newer, high-performance YOLO model. Finally, on the software side, it is necessary to develop code to manage failures: if a card crashes in the middle of training, this system would allow it to reconnect and resume directly from the round where it left off, without losing the progress of the other clients and without having to restart everything.